Implementation Plan: N8N Hands-On Intermediate Assessment

Topic 5: Loop Processing, Pagination & Rate-Limited Batching

Document Navigation

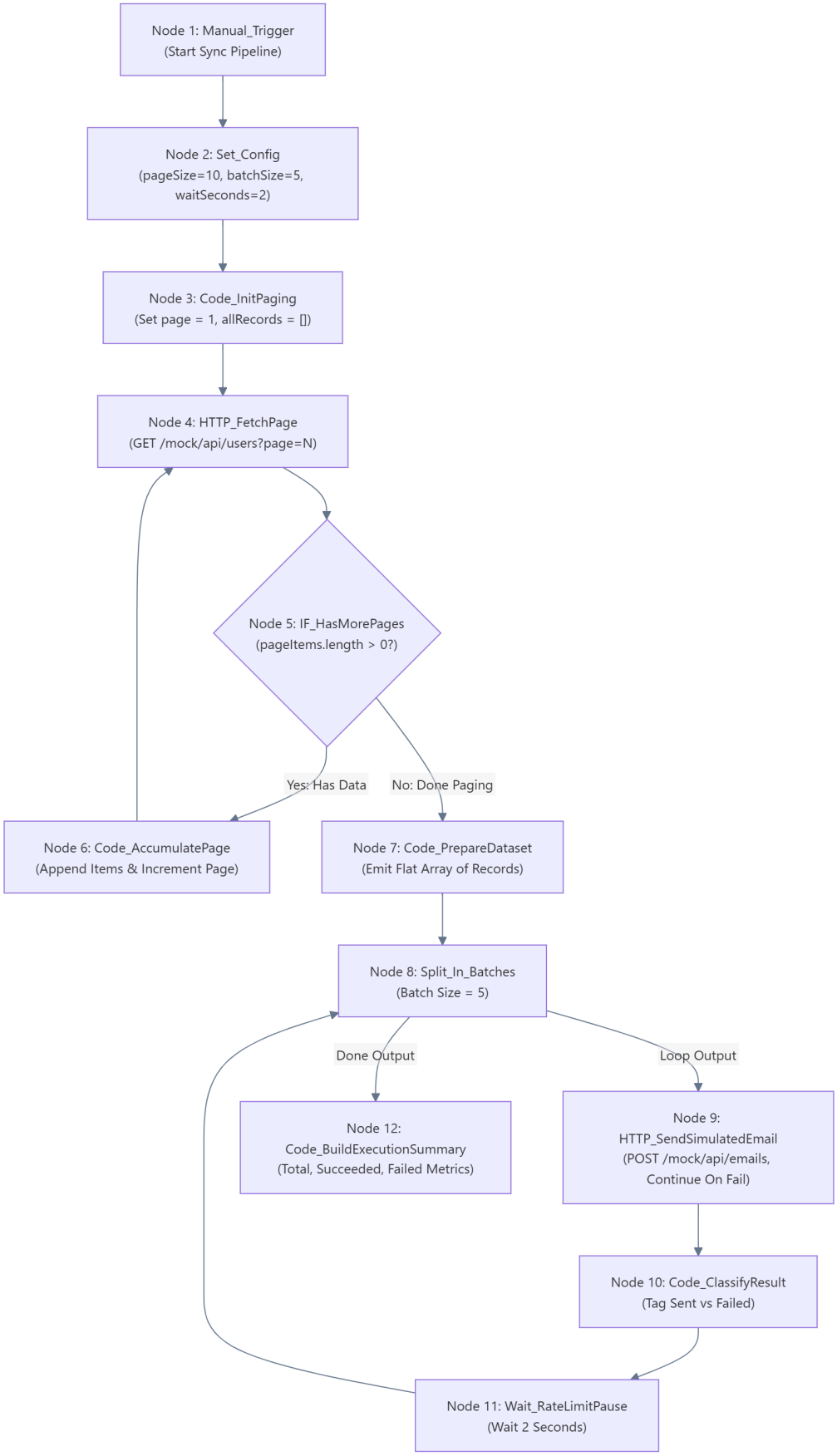
Concept Guide | Requirement Document | **Step-by-Step Implementation Plan** | Test Plan

1. Objective

Build an enterprise-grade paginated fetch and rate-limited batch processing workflow in n8n. The workflow loops through a paginated mock user API until all records are retrieved, consolidates the pages, splits the dataset into batches of 5, sends simulated emails with item-level error isolation, pauses for 2 seconds between batches, and aggregates final delivery statistics.

2. Canvas Architecture Overview

All rights reserved. Tayana Academy • Confidential Document



All rights reserved. Tayana Academy • Confidential Document

3. Prerequisites & Environment Setup

- [ ] n8n instance version 1.0.0 or higher.
- [ ] No external API keys or credentials needed (self-contained mock services included).

---

4. Step-by-Step Practical Build Guide

Step 1: Mock Paginated API & Dispatch Setup (Choose One of 3 Options)

To test pagination and rate-limited batching:

Option A: Free Cloud Mock via DummyJSON / JSONPlaceholder (Zero Install)

You can directly query free online REST endpoints that natively support pagination query parameters (`_page` and `_limit`):

- **Users Pagination Endpoint**: `https://jsonplaceholder.typicode.com/users` (or DummyJSON custom response via dummyjson.com/custom-response).
- **Email Simulation Dispatch Endpoint**: `https://jsonplaceholder.typicode.com/posts` (accepts POST requests and returns HTTP 201 Created).
- **Benefits**: No mock server setup required at all; query parameters `_page={{ $json.currentPage }}` and `_limit=5` work immediately.

Option B: Postman Mock Server (Multi-Page Examples)

1. In Postman, add a request `GET /api/users`.
2. Add Example 1 (Query Param `page=1`): return 10 user objects.
3. Add Example 2 (Query Param `page=2`): return 10 user objects.
4. Add Example 3 (Query Param `page=3`): return empty array `[]`.
5. Create a Mock Server and copy the URL: `https://{{mock-id}}.mock.pstmn.io/api/users`.

Option C: In-n8n Local Webhook Mock Server (Zero Dependencies)

1. Create a new workflow named `Mock Paginated API & Email Server`.
2. Add a **Webhook Node** named `Mock_Users_API` (`GET /mock/api/users`, Mode: Using Respond Node).
3. Connect a **Code Node** returning 23 mock users sliced by page:

```
const query = $input.first().json.query || {};
const page = Number(query.page || 1);
const limit = Number(query.limit || 10);

const allUsers = [];
for (let i = 1; i <= 23; i++) {
  allUsers.push({
    id: i,
```

All rights reserved. Tayana Academy • Confidential Document

```
    name: `User ${i}`,
    email: i === 13 ? "bad-email-format" : `user${i}@enterprise-test.io`,
    department: i % 2 === 0 ? "Engineering" : "Operations"
  });
}

const start = (page - 1) * limit;
const pageUsers = allUsers.slice(start, start + limit);
return { json: { items: pageUsers } };
```

4. Connect to a Respond node returning `={{ $json.items }}`.
5. Add another Webhook Node named `Mock_Email_API` (`POST` `/mock/api/emails`, Mode: Using Respond Node).
6. Connect an IF node checking for `@` in email (200 OK vs 400 Bad Request).
7. Save and activate the workflow.

Step 2: Configure Main Workflow Trigger & Constants (`Set_Config`)

1. **Create Workflow & Manual Trigger:**
   - In n8n, click **+ Add workflow** in the top-right corner.
   - Rename to: `Topic 5 - Loop Processing, Pagination & Rate-Limited Batching`.
   - Click **+ Add first step...**, search for **`Manual Trigger`**, and place it. Rename to: `Manual_Trigger`.
2. **Add Configuration Node:**
   - Hover over the right connector handle of `Manual_Trigger`, click **`+`**, search for **`Edit Fields`** (or `Set`), and select **Edit Fields (Set)**.
   - Rename to: `Set_Config`.
3. **Configure Parameters (Parameters Tab):**
   - **Mode:** Select **`Manual Mapping`**.
   - Under **Values to Set**, click **+ Add Field** to define execution parameters:
     1. **Name:** `pageSize` | **Type:** `Number` | Value: `10` (Fixed)
     2. **Name:** `batchSize` | **Type:** `Number` | Value: `5` (Fixed)
     3. **Name:** `waitSeconds` | **Type:** `Number` | Value: `2` (Fixed)
     4. **Name:** `maxPages` | **Type:** `Number` | Value: `10` (Fixed)
4. **Test Node Checkpoint:**
   - Click **Test step** and verify the 4 configuration constants appear in the Output pane.

Step 3: Initialize Paging State (`Code_InitPaging`)

1. **Add Node to Canvas:**

All rights reserved. Tayana Academy • Confidential Document

- Hover over `Set_Config` output handle, click `+`, search for **`Code`**, and add the node.
- Rename to: `Code_InitPaging`.

2. **Configure Parameters (Parameters Tab):**
   - **Language:** Select **`JavaScript`**.
   - **Mode:** Select **`Run Once for All Items`**.
   - **JavaScript Code:** Paste the initialization script:

```
// Global memory state for active pagination execution
const staticData = $getWorkflowStaticData('global');
staticData.accumulatedUsers = [];
staticData.currentPage = 1;

return [{
  json: {
    currentPage: 1,
    pageSize: $node['Set_Config'].json.pageSize || 10,
    status: "START_FETCH"
  }
}];
```

3. **Test Node Checkpoint:**
   - Click **Test step**. Verify `currentPage: 1` is returned.

Step 4: Fetch Paginated Records (**`HTTP_FetchPage`**)

1. **Add Node to Canvas:**
   - From `Code_InitPaging`, click `+`, search for **`HTTP Request`**, and add the node.
   - Rename to: `HTTP_FetchPage`.
2. **Configure Parameters (Parameters Tab):**
   - **Method:** Select **`GET`**.
   - **URL:** Toggle to **Expression (`fx`)**, enter:
     - *DummyJSON Option A:* `https://dummyjson.com/c/<your-id>`
     - *Postman Option B:* `https://{{mock-id}}.mock.pstmn.io/api/users`
     - *Local Option C:* `http://localhost:5678/webhook/mock/api/users`
   - **Send Query Parameters:** Switch toggle to **`ON`**.
   - Click **Add Parameter**:
     - **Name:** `page` | Mode: **Expression (`fx`)** | **Value:** `={{ $json.currentPage }}`
   - Click **Add Parameter**:
     - **Name:** `limit` | Mode: **Expression (`fx`)** | **Value:** `={{ $node['Set_Config'].json.pageSize }}`
   - **Authentication:** Select **`None`**.

All rights reserved. Tayana Academy • Confidential Document

3. **Test Node Checkpoint:**
   - Click **Test step**. Verify that a list of user objects is returned in the Output pane.

Step 5: Check For More Pages (`IF_HasMorePages`)

1. **Add Node to Canvas:**
   - From `HTTP_FetchPage`, click `+`, search for `If`, and place the node.
   - Rename to: `IF_HasMorePages`.
2. **Configure Parameters (Parameters Tab):**
   - Click **Add Condition**.
   - **Value 1:** Switch toggle to **Expression (`fx`)** and enter:

```
={{ Array.isArray($json.items) ? $json.items.length : ($input.all().length > 0 ? 1 : 0) }}
```

   - **Operation:** Select `Number` $ ightarrow$ `Larger than`.
   - **Value 2:** Enter `0` (Fixed).
3. **Test Node Checkpoint:**
   - Click **Test step**. When items exist, it evaluates to `true` (Top Branch). When empty, it evaluates to `false` (Bottom Branch).

Step 6: Accumulate Page & Loop Back (`Code_AccumulatePage`)

1. **Add Node to Canvas (Loop Branch):**
   - Hover over the **true (top) output handle** of `IF_HasMorePages`, click `+`, search for `Code`, and add the node.
   - Rename to: `Code_AccumulatePage`.
2. **Configure Parameters (Parameters Tab):**
   - **Language:** Select `JavaScript`.
   - **Mode:** Select `Run Once for All Items`.
   - **JavaScript Code:** Paste:

```
const staticData = $getWorkflowStaticData('global');
const inputData = $input.first().json;
const newItems = Array.isArray(inputData.items) ? inputData.items : $input.all().map(i => i.json);

// Accumulate items
staticData.accumulatedUsers = staticData.accumulatedUsers.concat(newItems);
staticData.currentPage += 1;

const maxPages = $node['Set_Config'].json.maxPages || 10;
const hasHitMax = staticData.currentPage > maxPages;

return [{
  json: {
    currentPage: staticData.currentPage,
```

All rights reserved. Tayana Academy • Confidential Document

```
    totalAccumulatedSoFar: staticData.accumulatedUsers.length,
    shouldContinue: !hasHitMax && newItems.length > 0
  }
}];
```

3. **Close the Pagination Loop:**
   - In the canvas, click and drag the output connector handle from `Code_AccumulatePage` back to the **input connector handle** of `HTTP_FetchPage`.
   - *Notice the animated loop line connecting them!*

Step 7: Prepare Full Consolidated Dataset (`Code_PrepareDataset`)

1. **Add Node to Canvas (Exit Branch):**
   - Click the **false (bottom) output handle** of `IF_HasMorePages` (fires when pagination finishes), click `+`, search for `Code`, and add the node.
   - Rename to: `Code_PrepareDataset`.
2. **Configure Parameters (Parameters Tab):**
   - **Language:** Select `JavaScript`.
   - **Mode:** Select `Run Once for All Items`.
   - **JavaScript Code:** Paste:

```
const staticData = $getWorkflowStaticData('global');
const users = staticData.accumulatedUsers || [];

// Unroll array into separate n8n items
return users.map(user => ({ json: user }));
```

3. **Test Node Checkpoint:**
   - Click **Test step**. Verify all 23 gathered user records are unrolled as separate items in the Output pane.

Step 8: Partition Dataset with Loop Over Items (`Loop_Over_Items`)

1. **Add Node to Canvas:**
   - From `Code_PrepareDataset`, click `+`, search for `Loop Over Items` (or `Split In Batches`), and select it.
   - Rename to: `Loop_Over_Items`.
2. **Configure Parameters (Parameters Tab):**
   - **Batch Size:** Toggle to **Expression (`fx`)** and enter:

```
={{ $node['Set_Config'].json.batchSize || 5 }}
```

All rights reserved. Tayana Academy • Confidential Document

3. **Test Node Checkpoint:**
   - Click **Test step**. Verify it partitions the 23 items into chunks of 5 items per iteration.

Step 9: Send Simulated Emails with Error Isolation (`HTTP_SendSimulatedEmail`)

1. **Add Node to Canvas:**
   - From the **loop output handle** of `Loop_Over_Items`, click `+`, search for `HTTP Request`, and add the node.
   - Rename to: `HTTP_SendSimulatedEmail`.
2. **Configure Parameters (Parameters Tab):**
   - **Method:** Select `POST`.
   - **URL:** Toggle to **Expression (`fx`)**, enter:
     - *DummyJSON Option A:* `https://dummyjson.com/c/<your-id>`
     - *Postman Option B:* `https://{{mock-id}}.mock.pstmn.io/api/emails`
     - *Local Option C:* `http://localhost:5678/webhook/mock/api/emails`
   - **Send Body:** Switch toggle to `ON`.
   - **Body Content Type:** Select `JSON`.
   - **Specify Body:** Select `Using JSON`.
   - **JSON:** Switch to **Expression (`fx`)** and enter:

```
{{ { "to": $json.email, "name": $json.name, "subject": "Enterprise Operations Update" } }}
```

3. **Configure Fault Isolation (Settings Tab):**
   - Click **Settings** tab.
   - **Continue On Fail:** Toggle to `ON` (critical: ensures one invalid email address does not crash the entire batch).
4. **Test Node Checkpoint:**
   - Click **Test step**. Verify response is received.

Step 10: Classify Result & Track Running Counts (`Code_ClassifyResult`)

1. **Add Node to Canvas:**
   - From `HTTP_SendSimulatedEmail`, click `+`, search for `Code`, and add the node.
   - Rename to: `Code_ClassifyResult`.
2. **Configure Parameters (Parameters Tab):**
   - **Language:** Select `JavaScript`.
   - **Mode:** Select `Run Once for Each Item`.
   - **JavaScript Code:** Paste:

All rights reserved. Tayana Academy • Confidential Document

```
const item = $input.item.json;
const isError = item.error !== undefined || item.statusCode >= 400;

return {
  json: {
    status: isError ? "FAILED" : "SENT",
    email: $input.item.json.to || $node['Loop_Over_Items'].json.email,
    processedAt: new Date().toISOString()
  }
};
```

3. **Test Node Checkpoint:**
   - Click **Test step**.

Step 11: Enforce Rate-Limit Delay (`Wait_RateLimitPause`)

1. **Add Node to Canvas:**
   - From `Code_ClassifyResult`, click `+`, search for `Wait`, and select the **Wait** node.
   - Rename to: `Wait_RateLimitPause`.
2. **Configure Parameters (Parameters Tab):**
   - **Resume:** Select `After time interval`.
   - **Wait Amount:** Toggle to **Expression (`fx`)** and enter:

```
={{ $node['Set_Config'].json.waitSeconds || 2 }}
```

   - **Unit:** Select `Seconds`.
3. **Close the Batch Loop:**
   - Click and drag the output connector handle from `Wait_RateLimitPause` back into the **input connector handle** of `Loop_Over_Items`.
   - *Notice the second loop line created on the canvas!*

Step 12: Aggregate Final Execution Metrics (`Code_BuildExecutionSummary`)

1. **Add Node to Canvas (Final Exit):**
   - Look at `Loop_Over_Items`. Notice its **done output handle** (fires after all batches complete).
   - Click the **done output handle**, click `+`, search for `Code`, and add the node.
   - Rename to: `Code_BuildExecutionSummary`.
2. **Configure Parameters (Parameters Tab):**
   - **Language:** Select `JavaScript`.
   - **Mode:** Select `Run Once for All Items`.

All rights reserved. Tayana Academy • Confidential Document

- **JavaScript Code:** Paste the final execution summary generator:

```
const staticData = $getWorkflowStaticData('global');
const totalUsers = (staticData.accumulatedUsers || []).length;

return [{
  json: {
    pipelineSummary: "BATCH_PAGINATION_COMPLETE",
    totalPagesHarvested: staticData.currentPage - 1,
    totalUsersProcessed: totalUsers,
    completedAt: new Date().toISOString()
  }
}];
```

3. **Test Node Checkpoint:**
   - Click **Test step**. Verify the final summary card reports total harvested pages and records processed.

5. Node-by-Node Settings Blueprint

Node ID	Node Name	Node Type	Critical Settings	Custom Expressions / Code
N5-01	`Manual_Trigger`	Manual Trigger	Trigger workflow on demand	N/A
N5-02	`Set_Config`	Set	`pageSize=10`, `batchSize=5`, `waitSeconds=2`	Static configuration tokens
N5-03	`Code_InitPaging`	Code	Initialize `currentPage=1` in static data	State initialization
N5-04	`HTTP_FetchPage`	HTTP Request	GET `/mock/api/users?page=N&limit=N`	`page: {{ $json.currentPage }}`
N5-05	`IF_HasMorePages`	IF	Check if received items array length > 0	`{{ $input.all().length }} > 0`
N5-06	`Code_AccumulatePage`	Code	Append items to array, increment `currentPage`	In-memory accumulator
N5-07	`Code_PrepareDataset`	Code	Emit flat array of items for batch processing	Flatten items
N5-08	`Split_In_Batches`	Split In Batches	Batch size: 5	`{{ $node['Set_Config'].json.batchSize }}`
N5-09	`HTTP_SendSimulatedEmail`	HTTP Request	POST to mock email endpoint, Continue On Fail ON	Dynamic email JSON body
N5-10	`Code_ClassifyResult`	Code	Classify item as `SUCCEEDED` or `FAILED`	Check error property
N5-11	`Wait_RateLimitPause`	Wait	Wait 2 seconds before next batch loop	`{{ $node['Set_Config'].json.waitSeconds }}`
N5-12	`Code_BuildExecutionSummary`	Code	Emit final sync summary object	Final audit summary

All rights reserved. Tayana Academy • Confidential Document

6. Verification Checkpoints & Testing Guide

Test 1: Full Pagination Loop Execution

1. Ensure the `Mock Paginated API & Email Server` workflow is active.
2. In the `Topic 5` canvas, click **Test Workflow**.
3. Verify `HTTP_FetchPage` executes 4 times:
   - Pass 1 (`page=1`): Returns 10 items.
   - Pass 2 (`page=2`): Returns 10 items.
   - Pass 3 (`page=3`): Returns 3 items.
   - Pass 4 (`page=4`): Returns 0 items (triggers loop exit).
4. Verify `Code_PrepareDataset` emits a consolidated list of exactly 23 items.

Test 2: Rate-Limited Batching Execution

1. Observe the `Split_In_Batches` execution:
   - Batch 1: 5 items processed $ ightarrow$ 2-second pause.
   - Batch 2: 5 items processed $ ightarrow$ 2-second pause.
   - Batch 3: 5 items processed $ ightarrow$ 2-second pause.
   - Batch 4: 5 items processed $ ightarrow$ 2-second pause.
   - Batch 5: 3 items processed $ ightarrow$ 2-second pause.
2. Confirm the loop terminates and routes to the **Done Output**.

Test 3: Fault Isolation Verification

1. Inspect record 13 (`User 13`, `email: "bad-email-format"`).
2. Confirm `HTTP_SendSimulatedEmail` received HTTP 400 from the mock server, but `Continue On Fail` prevented the workflow from aborting.
3. Confirm `Code_ClassifyResult` marked record 13 as `FAILED` while remaining records in the batch were marked `SUCCEEDED`.

---

7. Production Risks & Mitigation Strategies

Risk	Impact	Mitigation
Infinite Pagination Loop	API error causes endpoint to return static non-empty data indefinitely.	Implement maximum page guard (`currentPage > maxPages` exits loop).
Worker Thread Blocking	Wait node in high-volume environments holds worker resources.	Configure n8n with webhook/queue mode (`EXECUTIONS_MODE=queue`) using Redis.
Batch Memory Exhaustion	Aggregating 100,000+ items into static data crashes the container.	For massive datasets (>10k records), stream batches directly into a staging database table instead of in-memory arrays.

All rights reserved. Tayana Academy • Confidential Document